05 · Documento técnico

Cesta: listas de mercado que viram o carrinho mais barato do iFood, respeitando as preferências do usuário. Código em `Petroncini/Gnomers`.

Arquitetura

PWA em React conversa com um back-end FastAPI por REST, e o progresso da montagem chega por SSE. O back-end tem três partes com papéis separados:

- **LLM (Claude Haiku, via OpenRouter):** interpreta texto. Planeja as buscas de cada item, julga se cada produto serve ao pedido e converte mensagens do chat em preferências. Toda saída é JSON validado por esquema; se falhar, cai em regras fixas.
- **Otimizador (PuLP + CBC):** programação linear inteira que escolhe produtos e mercados. Não chama nada externo e dá o mesmo resultado para a mesma lista.
- **Camada do iFood:** toda chamada ao MCP passa por ela. Recusa ID que não veio de uma resposta anterior e qualificador na busca, guarda cache de leitura, normaliza os formatos de erro e grava as respostas para testes sem rede.

Fluxo "Montar carrinho" e uso do MCP

Etapa	O que faz	Tools
Endereço	GPS cruzado com os endereços salvos	`listAddresses` (sempre a 1ª)
Busca	até 3 buscas por item, planejadas pelo LLM; depois, busca nos mercados candidatos que não tinham o item	`searchOffers`, `getMerchant`
Juiz	cada produto vira exato, substituto ou não serve	nenhuma
Otimização	preço × embalagens + frete + penalidades, com pedido mínimo; só divide a compra se economizar mais que um limite	nenhuma
Carrinhos reais	as 3 melhores combinações viram carrinho; vence o menor total real	`createCart`, `setCartDelivery`, `listVouchers`, `setVoucher`, `setCartPayment`
Pedido	Pix e rastreio, respeitando `retryMessage` e `pollingStop`	`submitOrder`, `trackOrder`

Decisões principais

- **Busca e juiz pelo LLM.** A busca do iFood exige todas as palavras e não tem sinônimos, e o catálogo não tem marca, peso nem categoria como campos. O LLM gera as buscas, e um juiz decide, com o pedido inteiro, o que serve: "Zero Açúcar" atende "sem açúcar"; leite em pó não serve para "leite integral".
- **Preferências como regras tipadas,** com escopo (todas as listas ou uma), força (obrigatória ou preferência) e origem (usuário, chat, aprendida). Cada tipo age num ponto: dieta filtra a busca, marca bloqueada filtra produtos, marcas em ordem viram penalidade no otimizador, e "produto" vira a busca e o pedido do juiz.
- **Aprendizado por sinais:** escolher, aceitar, trocar e recusar. 3 sinais positivos e mais de 70% da categoria criam uma preferência de marca aprendida; a mediana dos intervalos entre compras dá a frequência de cada item.
- **Proatividade por evento:** uma promoção reavalia as listas e sugere adiantar (item perto de acabar, 15% ou mais de desconto) ou estocar (30% ou mais).
- **Agente com tools só no chat.** Montar o carrinho é um fluxo fixo, previsível e barato. No chat, onde os pedidos são abertos, um loop de tool use chama `searchOffers` e `getItem` e altera o carrinho real (`addItemToCart`, `updateCartItem`, `removeItemFromCart`). Limites: até 6 chamadas ao iFood por mensagem, só produtos vistos na conversa e nunca fecha pedido.

Robustez e medições

- 89 testes automatizados, sem rede, com um simulador do MCP; o contrato da API é testado dos dois lados.
- Renovação automática do token; no máximo 30 chamadas ao iFood por minuto; uma alteração por vez em cada carrinho.
- Em 42 listas com preços reais, o Cesta saiu 19% mais barato que comprar tudo no mercado mais perto. Em 275 casos marcados à mão, o juiz aceitou 6 produtos errados, contra 35 das regras fixas.

06 · Feedback sobre o MCP do iFood

Observado no sandbox entre 17 e 18/10 (São Carlos, contas de teste).

1. **Busca:** o `searchOffers` exige todas as palavras e não conhece sinônimos. "granola sem açúcar" e "granola zero açúcar" não têm nenhum resultado em comum, e com `sort: PRICE` "feijão" traz tempero antes do feijão. Sugestão: busca tolerante a sinônimos, termos de exclusão e marca, peso e categoria como campos de cada resultado.
2. **Limite de chamadas e busca em lote:** parte do volume é escolha nossa (3 buscas por item, para compensar a falta de sinônimos, e 3 carrinhos reais para comparar o preço final: 57 chamadas num carrinho medido, ~31 estimadas depois de otimizar). Mas o mínimo que a tool impõe é uma busca por item, uma de cada vez por sessão (1,2 a 1,8 s cada), e depois de ~60 chamadas em 1 a 2 minutos o token recebe `INTERNAL` em todas as tools, sem dizer que é limite nem quanto esperar. Sugestão: 429 com `Retry-After` (ou o limite na documentação) e várias consultas numa mesma chamada.
3. **Documentação divergente:** são quatro formatos de erro diferentes; o `createCart` funciona com o mercado fechado (a documentação diz que dá erro); os IDs de entrega reais são `FAST_DELIVERY/DEFAULT`; e o `cardTokenId` vem sempre nulo, então só Pix funciona.
4. **Faltando:** tools para apagar carrinho e endereço (comparar combinações deixa carrinhos abandonados).